



UNIVERSITY OF BUCHAREST

**FACULTY OF MATHEMATICS
AND COMPUTER SCIENCE**



COMPUTER SCIENCE PROGRAM

Bachelor's Thesis

INLINING COMPILER OPTIMIZATION THROUGH MACHINE LEARNING TECHNIQUES

Graduate

Ingrid-Adriana Corobană

Scientific Advisor

Prof. Dr. Cristian Rusu

Bucharest, 2026

Abstract

In optimizing compilers, the problem of inlining has long been studied in order to increase performance across platforms while keeping the size of applications small. Choosing when to inline remains complicated because, in the context of a program, functions are interconnected through the call graph, and one decision can influence the structure of the program and the future possible optimizations. These choices can affect both the size of the intermediate representation and the final object file at the end of compilation.

In this thesis, the problem of inlining is treated through an offline learning approach. Feature vectors about call sites and general graph information are collected at the beginning of the compilation stage and are not updated after every decision, as an online model would do. The dataset contains C/C++ files from DCMTK, a varied public-source supplement selected for inlining relevance, and deterministic synthetic modules generated combinatorially in order to add controlled cases. The files pass through the Clang frontend with inlining turned off in order to obtain LLVM IR modules, from which call sites are extracted and feature vectors are constructed.

A set of deterministic heuristics, called teachers, is defined and run on the feature vectors of each module. This is done in order to assign inline/no-inline labels per call site. For each module, the teacher that reduces the rewritten IR size the most is selected, and its decisions become part of the supervised training set. Student models are then trained to imitate these selected teachers through behavior cloning, and their predictions are applied by rewriting call-site sections of LLVM IR. The reported run compiles 163 C/C++ sources, evaluates 150 LLVM modules with local call sites, and extracts 10394 call-site samples. On the held-out split, the selected-teacher reference reduces IR instruction count by 1.10% relative to never-inline, while the best held-out student reaches a 0.61% reduction. These results do not outperform LLVM’s production `-Oz` pipeline, but they show that ML policies can be used to study inlining optimization instead of relying only on fixed classical heuristics.

Rezumat

În optimizarea compilatoarelor, problema de inlining este de mult studiată pentru a păstra o balanță între dimensiunea aplicațiilor și performanța acestora. Problema alegerii contextului potrivit pentru inlining rămâne complicată, deoarece funcțiile sunt interconectate prin intermediul grafului de apeluri, iar o decizie poate influența structura programului și viitoarele posibilități de optimizare. Toate aceste decizii pot influența dimensiunea reprezentării intermediare, precum și a binarului final, la finalul etapei de compilare.

În contextul acestei teze, problema de inlining este tratată printr-o metodă de învățare offline, în care vectorii de atribute cu informații despre apelurile de funcții și graful de apeluri sunt extrași la începutul compilării, fără a se reactualiza la fiecare pas, precum ar funcționa învățarea online. Setul de date conține fișiere C/C++ din DCMTK și alte surse publice variate, alese pe criteriul relevanței pentru problema de inlining. Multiple surse sintetice au fost adăugate pentru a extinde setul de date și au fost generate combinatorial, pentru a se asigura diversitatea. Întregul set de date trece printr-un frontend Clang cu optimizarea de inlining dezactivată pentru a se obține reprezentarea intermediară LLVM corespunzătoare, de unde se extrag call site-urile și se construiesc vectorii de atribute corespunzători.

Se definește un set de euristici numite profesori, care se rulează pe vectorii de atribute ai fiecărui program pentru a se obține etichete de inline/no-inline per call site. Profesorul care reușește să reducă numărul de instrucțiuni IR cel mai mult pentru un anumit program este folosit în setul pentru training supervizat. Setul de training este alcătuit astfel din reuniunea etichetelor profesorilor care au dus la cea mai mare reducere de IR per modul. Scopul este ca un student să fie capabil să copieze alegerile profesorilor săi, o tehnică numită behavior cloning. Deciziile studenților sunt aplicate prin rescrieri ale reprezentării intermediare la nivelul apelurilor de funcții. În experimentul principal, 163 fișiere C/C++ au fost compilate cu succes, dintre care 150 aveau apeluri locale de funcții și au rămas în setul de date. Numărul total de apeluri analizate este 10394. Pe setul held-out, cel mai bun model a dus la o reducere de IR de 0.61% față de never-inline. Cel mai bun profesor, folosit ca referință selected-teacher, a dus la o reducere de 1.10%. Deși rezultatele nu sunt mai bune decât LLVM -Oz, implementarea demonstrează că politicile de ML pot fi folosite în locul euristicilor clasice în problema de inlining.

Contents

1	Introduction	1
1.1	Research Questions and Contributions	2
2	Compiler Background	3
2.1	Compilation Pipeline	3
2.2	LLVM and LLVM IR	4
2.3	Static and Dynamic Measurements	4
3	Function Inlining	5
3.1	Cases Where Inlining Helps	5
3.2	Cases Where Inlining Hurts	6
4	Problem Formulation	7
5	Related Work	9
5.1	Classical and Empirical Views of Inlining	9
5.2	Autotuning and Pre-MLGO Compiler Learning	9
5.3	MLGO	10
5.4	Offline Imitation Learning and BC-Max	10
5.5	Positioning Against Prior Work	10
6	Methodology	12
6.1	Dataset Summary	13
6.2	Teacher Heuristics	14
7	Implementation	16
7.1	Repository Structure and Execution	16
7.2	Rewrite Algorithm	17
8	Evaluation	19
9	Limitations	21
10	Conclusions	22
A	Additional Diagnostics	23
	Bibliography	25

Chapter 1

Introduction

The problem of choosing when to optimize during compilation remains difficult, because compilers try to keep outputs predictable across platforms while also minimizing code size. This has led to a preference for predetermined rules called heuristics, which estimate when an optimization is useful. The issue with normal heuristics is that they are not very adaptable, while choosing when to optimize depends on multiple factors that can change together with the call graph and with other compiler optimizations. This thesis studies inlining as one case where this situational complexity matters: one choice can influence which optimizations are possible next, the size of the code, and which other inlining choices can follow [1, 2].

Inlining is one representative example of this compiler problem. By substituting a call site with the instructions corresponding to the called function, several simplification possibilities can appear: removing call overhead, enabling devirtualization, propagating constants, removing dead code, and simplifying the callee using information known in the caller. Although in many cases useful, inlining can also lead to unnecessary code growth. To avoid this, production compilers use thresholds and cost models that try to balance the cases where code size increases too much with the cases where inlining reveals useful optimization opportunities [1, 2, 3].

In the proposed method, the starting point is represented by several deterministic heuristics called teacher policies. Models referred to as students are trained on these teachers, with the purpose of testing whether they can learn useful inlining rules from static call-site features. The pipeline keeps track of the inline/no-inline choices of all teachers, evaluates them per module, chooses the teacher whose rewrites give the smallest IR size, and uses that teacher’s choices as labels for training. The idea of using feature vectors follows MLGO, while this prototype uses offline imitation learning instead of MLGO-style online reinforcement learning [4, 3, 5, 6].

The inlining of functions is a relevant case study, since it impacts both call-level properties and general, global program behavior. The call-level change is the choice to either substitute a call with the callee’s contents or to not edit the call site. The global effects appear through the changed structure of the call graph, the size of the caller, the possible redundancy of the callee, as well as new optimization options. This is reproduced in the implementation at a smaller scale: a set of teachers is run on every module, which results in inline/no-inline choices. In the case of the teacher per module that gives the biggest reduction of the LLVM IR size metric, those choices become labels for the training of binary classifier students.

For evaluation, LLVM IR is modified using LLVM’s `opt` tool. The number of LLVM IR

instructions after rewriting becomes the main metric for student-policy performance, while native object-file `.text` size is used as a secondary reference metric.

1.1 Research Questions and Contributions

ID	Research question	Thesis contribution
RQ1	Can inlining be framed as offline imitation learning from heuristics with a teacher role?	Formalizes inlining as a binary call-site policy problem with teacher heuristics, selected trajectories, and behavior cloning.
RQ2	Can BC-Max-style selection produce call-site labels under a code-size objective?	Evaluates 16 teachers per module and builds selected-teacher labels for supervised training.
RQ3	Can simple students imitate selected teachers from static features?	Implements the LLVM IR workflow for feature extraction, teacher design, behavior cloning, rewriting, size measurement, and student training.
RQ4	Do learned decisions reduce rewritten IR compared with never-inline?	Applies predictions through cloned callees, <code>alwaysinline</code> , LLVM <code>opt</code> , and held-out IR/native <code>.text</code> evaluation.
RQ5	What gaps remain relative to production ML-guided inlining?	Analyzes dataset scale, static rewrite approximation, teacher quality, and missing full LLVM inliner integration.

Table 1.1: Research questions and corresponding contributions.

Chapter 2

Compiler Background

2.1 Compilation Pipeline

The goal of compilation is to obtain a representation of code that can be executed on a machine. The intermediate representation, LLVM IR, and its corresponding instruction count are analyzed in this thesis because they are affected by inlining choices while still being more general than machine code. LLVM IR is generated before backend-dependent decisions, so it is useful for studying optimization choices before the final target-specific code is emitted. A simple version of an LLVM compilation pipeline is:

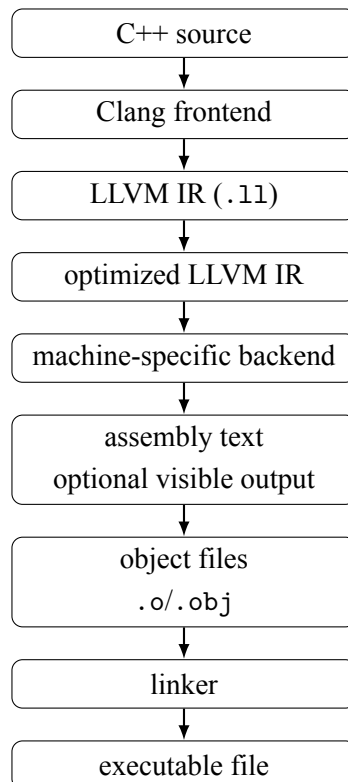


Figure 2.1: Simplified LLVM-based compilation pipeline.

Clang is used for checking syntax and converting C/C++ files into LLVM IR form. Following the optimization pass, the backend makes machine-dependent decisions, such as allocating virtual registers in the IR to real registers and writing the machine instructions that emit object files [7,

8]. LLVM IR is helpful for this implementation because it is more abstract than machine code but still lower-level than source code.

2.2 LLVM and LLVM IR

LLVM is a project for compiler infrastructure that includes optimization and backend tools, as well as libraries. LLVM IR is just one part of the project, corresponding to LLVM's implementation of an intermediate representation [9, 10]. It has an abstract version of register allocation called virtual registers, which get allocated at the backend stage, and also operations per variable type, memory-level operations, branches, and calls.

2.3 Static and Dynamic Measurements

In the context of this thesis, two concepts are used: static and dynamic call graphs, as well as static and dynamic code size. The static call graph contains all possible call choices, regardless of inputs, while the dynamic call graph is input-dependent and results from one execution of the code. Static code size refers to the dimension of a generated intermediate or final compilation representation, while dynamic code size is a measure of the executed instructions [7, 1].

Chapter 3

Function Inlining

Although inlining looks simplistic at a superficial level, when analyzing the problem in depth, it can be seen that one inline choice, through integrating the callee into the caller's context, can help further compact and optimize code by using known values or context-specific simplifications. The original call instruction may disappear, call overhead can be avoided, and later passes can optimize values and control flow across the merged code region [7, 8, 1].

3.1 Cases Where Inlining Helps

The examples given below were written for the purpose of this thesis. Small auxiliary functions can be simplified easily when inlined:

```
int ab(int a, int b) {                                int sum_ab(int a, int b) {
    return a * b;                                       return a + b + ab(a, b);
}                                                       }
```

Figure 3.1: Small helper and caller before inlining.

Through inlining, the call to `ab` is substituted with the multiplication returned by the function. This can improve the computation time needed for changing the call context, called call overhead, if `sum_ab` is executed frequently, meaning it is in a hot path.

When the function call contains constants, inlining can eliminate unused cases, sometimes by propagating constants. In the example below, inlining shows that `c` is 0 and minimizes code size since the `c == 1` branch can be eliminated:

```

// before:
string light_color(int c, int i) {
    if (c == 0)
        return light_red(i);
    if (c == 1)
        return light_green(i);
    return light_blue(i);
}

string car_color(int i) {
    int f = i;
    return light_color(0, 2 + f);
}

```

```

// after:
string car_color(int i) {
    int f = i;
    return light_red(2 + f);
}

```

Figure 3.2: Constant-propagation example before and after inlining.

3.2 Cases Where Inlining Hurts

Most recursive cases lead to big increases in code size, which is why most optimization algorithms penalize inlining here:

```

int sum(int x) {
    if (x <= 0)
        return 0;
    return x + sum(x - 1);
}

```

*// recursive inlining:
// exposes another recursive call
// repeated expansion
// needs recursion/growth checks*

Figure 3.3: Recursive call site and the reason it is usually rejected.

When a sequence in a program rarely gets executed, inlining can cause unnecessary growth of code:

```

void game_engine_reinit() {
    // large recovery path
}

```

```

void entry(bool error) {
    if (error)
        game_engine_reinit();
}

```

Figure 3.4: Inlining a large block of instructions in a rarely executed path puts pressure on the instruction cache and grows the static code unnecessarily.

When a bigger function gets called many times, its inlining can also expand static code significantly:

```

int get_info(int id) {
    int s = id;
    for (int i = 0; i < 1000; ++i)
        s = (s * 33) ^ i;
    return s;
}

```

```

void info_candidates() {
    cout << get_info(12) << endl;
    cout << get_info(133) << endl;
    cout << get_info(144) << endl;
    // can grow code unnecessarily
}

```

Figure 3.5: Large callee copied at multiple call sites if inlined aggressively.

Chapter 4

Problem Formulation

Consider P a program or module, and let

$$C(P) = \{c_1, c_2, \dots, c_n\}$$

be the collection of call sites collected in the order of parsing the program at inlining time. A call site represents places where functions are called in a program, and for each call site c_i , a feature vector x_i is extracted during compilation, with data about the caller, the function which initiates the call, and the callee, the function being called, as well as information about the call graph:

$$x_i = \phi(c_i),$$

where ϕ is the function that gathers features. In the context of the thesis, the features are static and composed of information about the caller and callee, such as their structure, length, functions that call them, how many constants they have, whether they are recursive, as well as some indices regarding the call graph.

In inlining, A is the binary set of possible actions:

$$A = \{0, 1\}, \quad 0 = \text{do not inline}, \quad 1 = \text{inline}.$$

To give insight into how good an action might be, a policy is used that gives a probability of action a for the vector of features x :

$$\pi_\theta(a \mid x)$$

where $\pi_\theta(a \mid x)$ denotes the likelihood of choosing action a , given feature vector x . Then, using $\arg \max$, the inlining choice is made:

$$d_\theta(x) = \arg \max_{a \in A} \pi_\theta(a \mid x).$$

Inlining gives the problem a chronological and interdependent nature. By applying policy π to program P , the following trajectory is output:

$$\tau_\pi(P) = ((x_1, a_1), (x_2, a_2), \dots, (x_T, a_T)),$$

where T represents how many inline/no-inline choices should be made for P . As previously discussed, through the possibility of affecting later optimizations, the decisions are dependent on each other [3, 6].

In the presented prototype, the vector of features is static and collected at the initial compilation stage from LLVM IR. This is done in an offline fashion, because the information is not updated after every inlining choice or change of the call graph, as an online inliner would do. The labels are predicted on this raw data and then the call sites are changed in LLVM IR, without runtime information.

The main scope is minimizing the size of the IR. In order to measure how well policies perform in the implementation, the count of IR instructions after rewriting is used:

$$\text{Size}_{IR}(P, \pi), \quad R(P, \pi) = -\text{Size}_{IR}(P, \pi).$$

The minus sign in the reward function is used because smaller values are desired.

Let

$$\mathcal{H} = \{h_0, h_1, \dots, h_k\}, \quad h^*(P_j) = \arg \min_{h_i \in \mathcal{H}} \text{Size}_{IR}(P_j, h_i)$$

be the set that includes all policies that act as teachers, and $h^*(P_j)$ is the selected teacher for a module P_j , which makes its IR size smallest. Every teacher heuristic outputs an inline/no-inline choice starting from information per call site.

The path taken by the best teacher gets included in the training set for the students:

$$\mathcal{D} = \bigcup_j \{(x_{j,t}, a_{j,t}^*)\}_{t=1}^{T_j},$$

where $a_{j,t}^*$ is the action that the chosen teacher policy $h^*(P_j)$ decided on for the vector of features $x_{j,t}$.

The new policy called a student should theoretically aim to predict labels as closely as possible to its teachers, therefore minimizing a disagreement function:

$$\min_{\theta} \sum_{(x,a) \in \mathcal{D}} \mathbf{1}[\pi_{\theta}(x) \neq a], \quad \mathcal{L}(\theta) = - \sum_{(x,a) \in \mathcal{D}} \log \pi_{\theta}(a | x).$$

The practical implementation uses a more direct approach: standard classifiers. For probabilistic classifiers, this can also be written through binary cross-entropy in order to obtain a probability.

The newly trained student can be used as a new policy and added to the teacher set. It gives inline/no-inline predictions per feature vector corresponding to a call site:

$$\hat{a} = \arg \max_{a \in \{0,1\}} \pi_{\theta}(a | x).$$

What the trained model predicts leads to rewrites in LLVM IR, and the number of instructions after rewriting is used as the performance metric.

Chapter 5

Related Work

5.1 Classical and Empirical Views of Inlining

Before ML models, this difficult problem was tackled using classical inliners. These attempted to find a compromise between code expansion and the chance for future code simplifications when choosing whether or not to remove call sites. Theodoridis, Grosser, and Su discuss how complex optimal inlining becomes when weighing the impact on consecutive chains of function calls, code size, and following optimization passes [2]. Godbolt gives a similar interpretation for the use of inlining: it can merge code by using known values, eliminate cases that are not needed, and simplify the callee with information known in the caller [1, 11].

5.2 Autotuning and Pre-MLGO Compiler Learning

The idea of empirical search, as well as learning approaches, preceded MLGO. MILEPOST GCC, for instance, constructs vectors of features in order to make the optimization process more adaptive [4]. This was helpful in the methodological sense for the thesis, since the concept of feature vectors was reproduced, and it demonstrated that empirical data can be used for the training of policies, even though its focus was general compiler optimization, not inlining in particular.

The issue of having fixed predetermined parameters in inlining rules was tackled by Cavazos and O’Boyle. They showed useful results by using empirically adjustable parameters in place of static ones [12]. The problem they approached is similar to the one proposed here, but with another scope, since they tried to tune policies automatically, while here the policy is trained on other policies. The technical difference is therefore that autotuning tries to find more appropriate parameters for heuristics, while behavior cloning attempts to train a decision rule starting from feature vectors at call-site level.

The focus on code size, specifically LLVM IR, is intentional in this thesis. It is an easy-to-calculate and objective metric. Object size, also easy to calculate, is listed secondarily. In order to assess real-world performance, other metrics such as runtime have been attempted in the literature, despite their difficulty to measure objectively and their dependence on inputs and target platforms. In recent work, MLGOPerf uses ML techniques modified toward minimizing both code size and performance metrics [13].

5.3 MLGO

MLGO was the initial inspiration for this thesis, because of the idea of using trained models and their predictions for modifying LLVM IR with the purpose of minimizing size [3, 5]. The reinforcement-learning model in MLGO uses states composed of call-graph indicators and call-site data, actions which can be either inlining or not, and a reward showing how well the final compiled program improved, usually through native code-size reduction. Using feature vectors avoids the computational burden of capturing all characteristics of the call graph. MLGO applies a policy-gradient technique, which is expensive because each change needs to be recompiled and the reward can be calculated only at the end [3].

5.4 Offline Imitation Learning and BC-Max

Marinov, Agarwal, and Trofin show that computationally expensive online reinforcement learning is not the only well-performing approach for this problem. They propose using an offline model trained on predefined teachers through imitation learning [6]. For the scope of inlining, let:

$$\mathcal{H} = \{h_0, h_1, \dots, h_n\}$$

represent the collection of heuristic teachers, and let P_i be a module with corresponding $\text{Size}_{IR}(P_i, h)$, meaning the size of the rewritten IR with teacher h .

The selected teacher per module has the smallest size of this rewritten IR:

$$h_i^* = \arg \min_{h \in \mathcal{H}} \text{Size}_{IR}(P_i, h).$$

The trajectory corresponding to the selected heuristic’s choices is the following:

$$\tau_i = \{(s_1, h_i^*(s_1)), \dots, (s_m, h_i^*(s_m))\}.$$

Here, s_t corresponds to the feature vector for call site number t . Here, the learning is done in a supervised way, from the labels of the union of the selected teachers, rather than by means of exploration and recompiling.

What teachers label individual call sites is context-dependent and sometimes does not generalize. Two different teachers might assign different labels to similar call sites in different modules, leading to disagreements in the set used for training the behavior-cloning students. This is why the actual rewrite impact is the real measure of success, not only how well the students agree with teachers.

5.5 Positioning Against Prior Work

Table 5.1 summarizes how the prototype borrows the policy-learning view from MLGO and BC-Max while remaining smaller and more inspectable than production compiler infrastructure.

Work	Main idea	Evaluation focus	Relation to this thesis
Cavazos and O’Boyle [12]	empirical tuning of inliner parameters	benchmark performance	early evidence that inlining choices can be optimized empirically
MILEPOST GCC [4]	program-feature-based optimization selection	empirical benchmark performance	broad compiler ML rather than call-site-level inlining
MLGO [3, 5]	learned policies integrated inside LLVM	code size and performance on large corpora	main learned-heuristic reference, not reproduced here
BC-Max [6]	offline imitation learning from multiple baselines	final objective value	closest template for teacher selection and behavior cloning
Theodoridis et al. [2]	autotuned search for high-reduction inlining-for-size decisions	native binary size on SPEC2017, LLVM, and SQLite	motivates code-size evaluation and stronger baselines
This thesis	BC-Max-style supervised pipeline over static call-site features	rewritten IR and native text-section reference	inspectable offline pipeline for call-site-level policy experiments

Table 5.1: Related-work positioning for ML-guided and empirical inlining optimization.

Chapter 6

Methodology

The implementation follows Figure 2.1 until the output LLVM IR step, whose size is the main objective to minimize. For every individual call site, a feature vector is produced, the set of teachers is run to select the best one per module, students are trained on those labels, and the labels predicted by the student classifiers are reapplied to the IR using LLVM opt. How sources are collected, how IR is generated, how teachers and students are implemented, how IR is rewritten, and how IR size is determined remain visible in the repository.

The dataset contains C/C++ files from various sources, including a small DCMTK snapshot¹, 76 supplements from public sources selected with inlining-relevance criteria, and synthetic sources generated deterministically from a $4^6 = 4096$ configuration space. For the generated sources, the format, content, and size of callees and callers can vary, as well as the order of calls and the arguments. This creates different function sizes, branches, mathematical functions, recursion, constants, and variables. The 70 generated modules contain 1120 call sites, so teachers are run on both representative inline and no-inline scenarios. Extending transferability beyond this generated set remains future work.

Files from public sources are filtered using the criteria of successful compilation to LLVM IR, the existence of a minimum of one local call site, and the ability to pass the teacher-selection step and the corresponding IR rewriting process. These steps happen at the data-collection stage, before training the students. The features in the prototype represent local-level approximations of the publicly available features used by MLGO: information about the call site, the caller and callee, and the general program-level call graph, with flags for recursive functions, constants, number of calls, and number of instructions.

¹DCMTK source repository: <https://github.com/DCMTK/dcmtk>.

Feature group	Implemented fields	Type	What the values indicate
Caller shape	caller-prefixed counts: basic blocks, conditional blocks, users, IR instructions, and call instructions	numeric	size and branching of the function that receives the inlined body; large or call-heavy callers may already be expensive to grow
Callee shape	callee-prefixed counts: basic blocks, conditional blocks, users, IR instructions, and call instructions	numeric	estimated body copied by inlining, plus whether the callee has few users and may become removable after all useful calls are inlined
Call-site context	callsite_height, cost_estimate, and number of constant parameters	numeric	local decision context: approximate graph depth, proxy inline cost, and constants that may enable simplification after the callee body is copied into the caller
Module graph	module edge count and node count	numeric	rough module-level density, used to distinguish sparse files from call-heavy modules where one decision can interact with many nearby calls
Safety signal	recursive-call indicator	binary	marks recursive expansion patterns, where repeated inlining can grow code unnecessarily

Table 6.1: Grouped feature schema used by all teachers and students. The complete ordered feature list is fixed in `scripts/data.py`.

6.1 Dataset Summary

The current prototype uses a DCMTK-based source snapshot, generated modules, and a public-source supplement in order to test the offline imitation-learning pipeline on a nontrivial number of call sites.

Quantity	Value
Source mix	DCMTK, generated, 76 public files
Candidate C/C++ files	164
Compiled C/C++ files	163
Modules with local call sites	150
Extracted call sites	10394
Train samples	9009
Hold-out samples / modules	1385 / 26
Teacher policies / students	16 / 7
Primary metric	rewritten IR instructions
Secondary metric	native .text size

Source group	Teacher IR red.	Student IR red.
Generated	8.76%	2.75%
Real source	1.22%	1.08%
All modules	1.66%	1.18%

Table 6.2: Dataset summary and source-group IR reduction. “Teacher” means the selected-teacher reference; “Student” means the best full-set student conservative_forest. The most frequent selected teachers are small_callee on generated modules and greedy_ir_size on real-source modules.

At call-site level, 9128 of the 10394 examples come from public source files. The train/test split is performed by module rather than by individual call site, so all call sites from one module stay on the same side of the split. This avoids testing on contexts that are nearly duplicated from the same caller, callee, or call graph.

Candidate group	Files	Compiled	Reason
DCMTK snapshot	18	17	source-level local call sites
Generated modules	70	70	controlled inlining and no-inlining patterns
Public sources	76	76	compiled C/C++ files with local call-site signal

Table 6.3: Dataset accounting. Filtering is applied before student training and keeps source-level modules with local call sites relevant to the rewritten-IR objective. Public-source provenance, licenses, and selected-file metadata are kept in the repository manifests and source snapshot files.

6.2 Teacher Heuristics

In this thesis, a teacher heuristic means a deterministic policy used to produce labels for student training. The local teacher set is intentionally simple and inspectable. The teachers use deterministic rules starting from the same call-site-level feature vector. Let s be callee size, u callee users, b branch count, k constant arguments, and r a recursion flag. Most rules use the following general shape:

inline if $B(x) - C(x) > 0$, $B(x) \approx 2k + \mathbf{1}_{u=1} + \text{depth signal}$, $C(x) \approx s + b + \text{nested calls} + r$.

The exact thresholds are fixed in `teach.py`; the table gives the readable form of each rule. The exception is `greedy_ir_size`, whose labels are produced by measured rewrites in `rewrite.py`. Therefore, most teachers are cheap proxy policies, while the greedy heuristic is slower because it actually tests candidate rewrites.

Teacher	Short rule
never_inline	Conservative baseline: never inline.
small_callee	Inline if s is small and $r = 0$.
single_caller	Inline if $u = 1$ and the callee is not large; the standalone helper may disappear after cleanup.
constant_argument	Inline if $k > 0$, so caller-known constants may remove branches or simplify expressions.
cost_budget, growth_budget	Inline if estimated growth $G \approx s \max(1, u) + b$ + nested calls stays under a budget.
loop_averse, hot_leaf	Prefer leaf-like callees; reject large, branchy, or call-heavy callees.
balanced_score, llvm_like_size, benefit_cost_ratio	Inline if $B(x) / \max(1, C(x))$ passes a threshold.
aggressive_speed	Accept more small and medium calls, while still rejecting recursive or clearly large cases.
ml_linear_score	Inline if $w \cdot x + \beta > 0$, using fixed hand-written weights rather than training.
bandit_ucb_proxy	Inline if $\hat{g}(x) + \alpha / \sqrt{1 + u} > 0$: estimated gain plus a small uncertainty bonus.
rl_value_proxy	Inline if immediate simplification plus future call-graph value exceeds growth penalty.
greedy_ir_size	Measured teacher: order plausible non-recursive candidates, try candidate rewrites, and keep a candidate only when the actual rewritten IR instruction count decreases.

Table 6.4: Teacher policies as readable decision rules.

The ML-, bandit-, and RL-inspired teachers are approximations rather than actually trained teachers. The measured greedy teacher is more expensive because it orders plausible candidates from static features, then uses the same clone-and-alwaysinline rewrite path as evaluation and keeps only locally improving rewrites. This distinction matters because the student does not learn from a single oracle; it learns from the teacher that behaves best for each module under the rewritten-IR objective.

The main reported run uses one offline teacher-selection round: teachers are evaluated, one teacher trajectory is selected per module, and students imitate those selected labels. Chapter 8 also reports a small three-round check in which the best learned student is fed back as another measured candidate policy.

Chapter 7

Implementation

The main pipeline and implementation are located in the repository root. The implementation generates synthetic modules, uses compilation stages to reach LLVM IR, extracts call-site features, evaluates teacher policies, trains students from the best-performing teachers, rewrites selected call sites, and writes artifacts that can be inspected. The source repository is available at:

<https://github.com/dirgnic/optimizingInlining>

7.1 Repository Structure and Execution

The scripts cover generation, feature extraction, teacher selection, student training, rewriting, and figure generation. After cloning the repository, the full pipeline can be run from the repository root with `python3 scripts/run.py all`. A short inspection run is available as `python3 scripts/run.py demo`; it compiles a few generated modules, prints the extracted call-site order and inline/keep decisions, and writes before/after demo IR under `out/demo/rewritten/`. The reported numbers come from deterministic source generation, a small number of DCMTK files, and a fixed module-level train/test split, with artifacts written under `out/`.

The default execution path is organized as shown in Table 7.1. Only scripts used by the reported run are listed here.

Step	Script	Role
0	<code>data.py</code>	defines shared directories, feature names, and call-site data structures.
1	<code>run.py</code>	coordinates the full command <code>python3 scripts/run.py all</code> and the short demo command <code>python3 scripts/run.py demo</code> .
2	<code>build.py</code>	generates the deterministic synthetic modules, compiles C/C++ sources to LLVM IR, and measures native compiler baselines.
3	<code>ir.py</code>	parses LLVM IR and extracts feature vectors for local call sites.
4	<code>teach.py</code> and <code>rewrite.py</code>	apply teacher rules, rewrite IR with LLVM opt, and select the best teacher per module by rewritten IR instruction count.
5	<code>teach.py</code>	converts the selected teacher actions into the behavior-cloning dataset.
6	<code>learn.py</code>	trains the student classifiers and stores their inline/no-inline predictions.
7	<code>rewrite.py</code>	applies student predictions through the same clone-and-opt rewrite path, then measures rewritten IR and native text-section size.
8	<code>graph.py</code> , <code>paper.py</code> , and <code>visual.py</code>	write the call-graph trace, generated Markdown summary, and figures used for inspection and reporting.

Table 7.1: Default script order for the reported thesis pipeline.

The local run uses macOS on Apple Silicon, Homebrew Clang/LLVM 15.0.7, and Python 3.9.6. Source files are translated to LLVM IR using the Clang frontend with a strict no-inlining configuration, in order to observe the call-graph structure before premade inlining optimizations. Reproducibility comes from deterministic generation, fixed model seeds in `learn.py`, and a deterministic module-level split.

7.2 Rewrite Algorithm

The rewriting algorithm works at call-site level and applies an LLVM IR transformation. For each selected call site, the callee is cloned, the selected call is redirected to the clone, and LLVM performs inlining and cleanup. Recursive or unresolved cases are skipped. This step is not the actual LLVM inliner, but a smaller reproducible inlining mechanism built for the experiment. During evaluation, inline predictions trigger this rewrite path, while no-inline predictions leave the call site unchanged; the final metric is therefore measured after predictions are materialized in LLVM IR.

The call-site order is deterministic and textual. Modules are analyzed in sorted .ll filename order. Within each module, functions are visited in the order in which their LLVM `define` blocks appear. Within each function, call instructions are scanned from top to bottom. This is not dynamic execution order and not a topological call-graph order. Non-greedy teachers and student policies keep this extracted order when preparing rewrites. The measured `greedy_ir_size` teacher is the exception: it first filters plausible non-recursive candidates, orders them by benefit/cost score, and

accepts a candidate only when the actual rewritten IR instruction count decreases.

Student model	Role in the experiment
<code>logistic_regression</code>	Linear, interpretable classifier.
<code>decision_tree</code>	Small threshold tree over individual features.
<code>knn</code>	Copies labels from nearby feature vectors.
<code>random_forest</code>	Ensemble of shallow trees.
<code>deep_forest</code>	Forest with deeper trees, giving a more flexible rule set.
<code>conservative_forest</code>	Forest variant with a higher effective inline threshold.
<code>small_mlp</code>	One-hidden-layer neural classifier.

Table 7.2: Student model families used for behavior cloning.

The student models do not discover useful inlining decisions by recompiling programs or exploring random actions during training. They receive the same static call-site feature vectors and learn, through behavior cloning, to minimize disagreement with the selected teacher for each module. Their quality therefore depends on both parts of the setup: whether the teacher set contains useful trajectories and whether the chosen model family can generalize those trajectories to held-out modules. This is why several simple classifiers are used rather than a single learner; each one represents a different compromise between interpretability, flexibility, and risk of copying noise from the selected labels.

Chapter 8

Evaluation

In the evaluation stage, 163 C/C++ sources are compiled, resulting in 150 LLVM modules with local call sites and 10394 call-site samples. The deterministic module split uses 9009 samples from 124 modules for training and 1385 samples from 26 modules for held-out testing. Teacher-label match measures agreement with selected-teacher labels; the actual code-size effect is measured only after predicted rewrites are applied.

In the teacher-evaluation and selection stage, every teacher is run on each module in order to determine which one produces the smallest rewritten IR. On held-out modules, this selected teacher is an oracle-style reference because it is chosen after measuring all candidate teachers on those modules; it is not a single deployable policy. Across all 150 modules, average rewritten IR falls from 1985.39 instructions for never-inline to 1952.39 for this reference, a 1.66% reduction. On the held-out split, the corresponding reduction is 1.10%: 1569.23 average IR instructions for never-inline versus 1552.04 for the selected teacher.

Model	Teacher-label match	Held-out Avg. IR	IR reduction	Gap to teacher
logistic_regression	0.901	1569.69	-0.03%	17.65
decision_tree	0.944	1565.50	0.24%	13.46
knn	0.914	1588.08	-1.20%	36.04
random_forest	0.952	1562.46	0.43%	10.42
deep_forest	0.955	1559.62	0.61%	7.58
conservative_forest	0.947	1562.69	0.42%	10.65
small_mlp	0.931	1607.35	-2.43%	55.31
deep_forest per-module: median 0.17%, min -0.52%, max 21.76%, 18 improved, 7 same, 1 worse.				

Table 8.1: Held-out student results on the 26 test modules. Teacher-label match measures imitation of selected-teacher labels; IR reduction and gap to selected teacher measure the rewritten-IR result.

Teacher-label match is only a behavior-cloning diagnostic. The IR size is measured only after the predicted rewrites are applied and the instruction count is recomputed. `small_mlp` matches labels reasonably well but still grows IR, while `deep_forest` is closest to the selected-teacher reference in rewritten IR size. Similar label agreement can therefore lead to different code-size behavior.

The student families, thresholds, and hyperparameters are fixed in the scripts before this evaluation. The phrase “best held-out student” is descriptive: it reports which fixed model had the

smallest held-out rewritten IR in this run. No additional hyperparameter search or model-family selection is performed on the held-out split.

Policy	Held-out Avg. IR	IR reduction
never_inline	1569.23	0.00%
attempted always-inline	1995.27	-27.15%
selected teacher	1552.04	1.10%
deep_forest	1559.62	0.61%
conserv. forest	1562.69	0.42%

Table 8.2: Held-out reference comparison.

Native workflow	.text bytes	Obj. bytes
no_inline_0z	674660	2225096
llvm_0z	574044	1585120
Reduction	14.91%	28.76%

Table 8.3: Compact LLVM -0z native-size comparison.

Scope	Teacher IR red.	Student IR red.
All 80	1.22%	1.08%
13 held-out	0.32%	0.19%

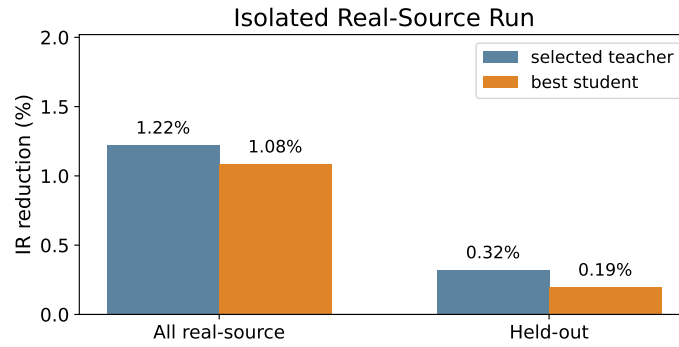


Table 8.4: All-real-source isolated run. The experiment retrains and rewrites only real-source modules; the plot visualizes the same reductions as the table.

Positive reductions mean smaller rewritten IR than never-inline. The best held-out student is `deep_forest`, with 1559.62 average IR instructions, a 0.61% reduction, and a 7.58-instruction gap to the selected teacher. On the full artifact set, `conservative_forest` is best, reducing all modules by 1.18%. The held-out reductions remain small because many real-source modules leave little profitable local inlining margin and because features are not recomputed after each accepted rewrite.

Table 8.3 frames the learned policies as prototype results rather than production replacements. On held-out rewritten objects, `deep_forest` reduces average .text from 5755.23 to 5705.23 bytes, but LLVM -0z remains the stronger native-size baseline. A three-round BC-Max-style check fed the previous best student back into the candidate pool, but the held-out result plateaued: the reference stayed at 1.10% and `deep_forest` stayed at 0.61%.

Chapter 9

Limitations

- **Dataset scale.** One of the most important limitations is the scale of the dataset. In the main experiment, 163 C/C++ files were successfully compiled, and 10394 call-site samples were extracted, including examples from 76 public-source files. Even though this is enough for the implemented prototype, stronger generalization would require larger benchmark families, such as SPEC, cBench/MiBench, SQLite builds, and wider LLVM test-suite subsets.
- **Held-out scope.** The held-out split contains 1385 samples from 26 unseen modules. However, all modules still come from the same general source collection and generator regime. A stronger future evaluation would use modules from more clearly different sources, in order to test whether the learned students generalize outside the current dataset composition.
- **Rewrite approximation.** In the main experiment, the feature vectors are extracted once from the initial LLVM IR. The chosen call sites are then rewritten by cloning selected callees, marking them with `alwaysinline`, and letting LLVM `opt` perform the actual inlining and cleanup. This is an offline approximation: the call graph and the feature vectors are not recomputed after every accepted decision, as they would be in a fuller online inlining system closer to LLVM’s own inliner.
- **Metric scope.** The main scope of the prototype is minimizing rewritten LLVM IR instruction count. Native `.text` size and object-file size are used as secondary measurements, but final linked executable size and runtime behavior are outside the current evaluation. Therefore, the results should be interpreted as IR-level and artifact-level evidence, not as a complete performance evaluation.
- **Teacher and MLGO scope.** BC-Max-style selection can only choose among the available candidate policies. In this thesis, the teachers are deterministic and inspectable proxy heuristics, not production inliner policies. A full reproduction of MLGO would require deeper integration into LLVM, stronger learned or production-level teachers, and a much larger corpus.
- **Iterative scope.** A three-round BC-Max-style check was also attempted, where the best learned student was fed back into the set of measured candidate policies. This did not lead to larger improvements in the reported run. Moreover, this check does not include the uncertainty-based trajectory exploration used in larger BC-Max-style systems.
- **Generated-data filtering.** The generated modules are useful because they add diverse and controlled inlining cases, but they are still synthetic. A future generator could add a discriminator that rejects near-duplicate modules and keeps rarer or more informative inlining patterns before training.

Chapter 10

Conclusions

The purpose of this thesis was to study how compiler inlining can be optimized using offline methods for learning policies, and to implement a reproducible end-to-end pipeline that works with LLVM IR. In the reported run, 163 C/C++ sources are compiled successfully, 150 LLVM modules with local call sites are evaluated, 16 teacher heuristics are run, 7 student models are trained, and the predicted inline/no-inline decisions are applied back to LLVM IR through call-site-level rewrites.

The main held-out result is measured on a source-level mix composed of DCMTK files, generated modules, and 76 public C/C++ source files. On the 26 held-out modules, the selected-teacher reference reduces the average rewritten LLVM IR instruction count from 1569.23 to 1552.04, which corresponds to a 1.10% reduction relative to the never-inline teacher. For the student models, the best held-out result by IR size is obtained by `deep_forest`, which reaches 1559.62 average IR instructions, corresponding to a 0.61% reduction. On the full artifact set, `conservative_forest` reaches 1961.94 average instructions and reduces rewritten IR by 1.18%.

The comparison with LLVM `-Oz` shows that the prototype should not be interpreted as a production-ready replacement for LLVM’s full optimization pipeline. On the standalone C/C++ files, LLVM `-Oz` reduces native `.text` size by 14.91% and object-file size by 28.76% relative to `no_inline_Oz`. The learned policies therefore demonstrate the workflow and problem formulation, rather than outperforming LLVM’s production size pipeline.

The major contribution of the thesis is methodological. The implementation connects source collection, LLVM IR generation, feature extraction, teacher evaluation, behavior-cloning dataset construction, student training, rewrite application, and artifact-level measurement in one reproducible workflow. The results show that call-site-level inlining can be framed as an offline policy-learning problem, where students imitate the best available teacher trajectories and their predictions are then tested through actual LLVM IR rewrites. Stronger future results would require integration with LLVM’s real inliner, larger real benchmark suites, stronger teacher policies, and measurements on final linked binaries.

Appendix A

Additional Diagnostics

The generated out/ directory contains the full JSON outputs used for inspection. Figures A.1 to A.5 keep the main diagnostics in portrait layout, with readable labels and values.

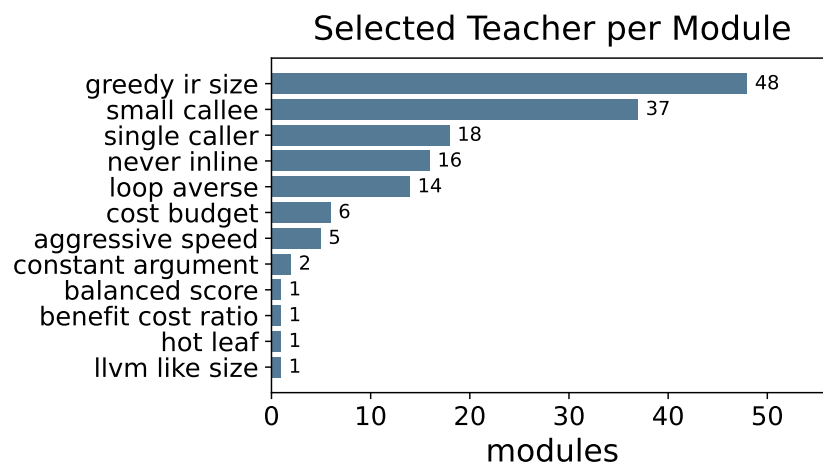


Figure A.1: Selected teacher distribution by module.

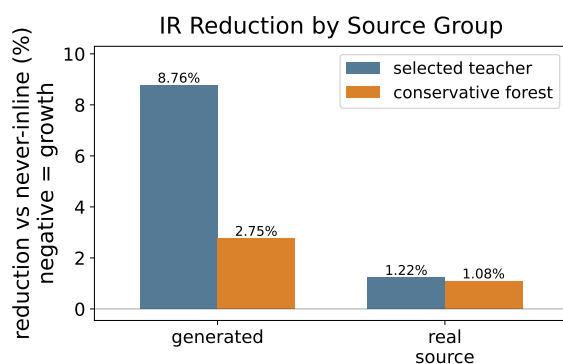


Figure A.2: Teacher and student IR reduction by source group.

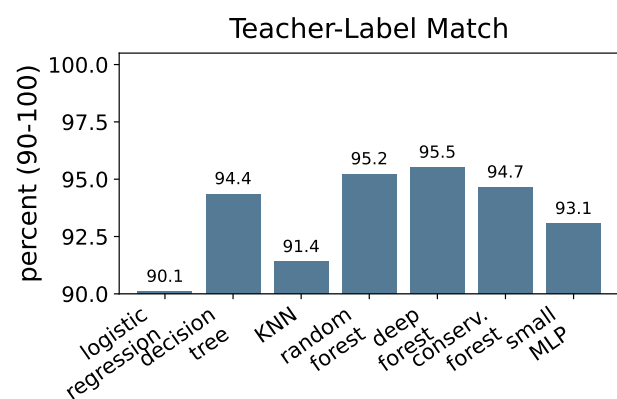


Figure A.3: Held-out teacher-label match for student models.

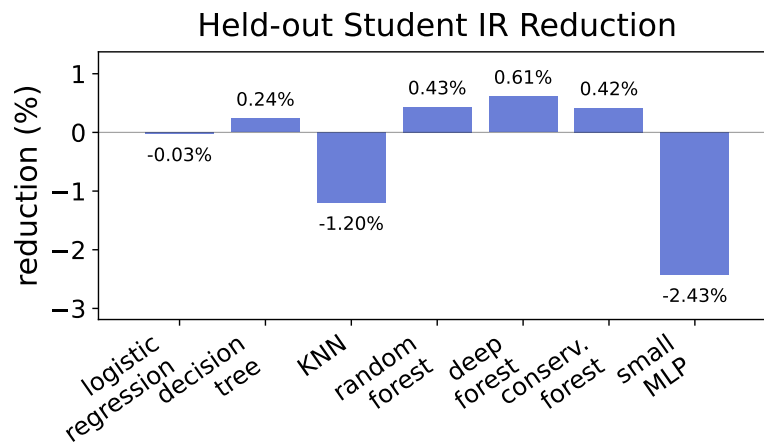


Figure A.4: Held-out student IR reduction.

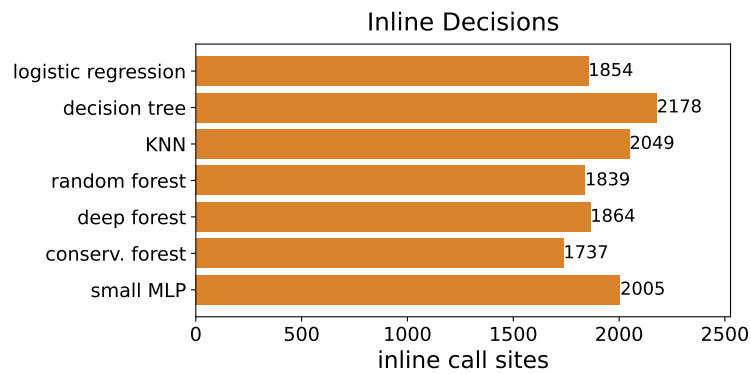
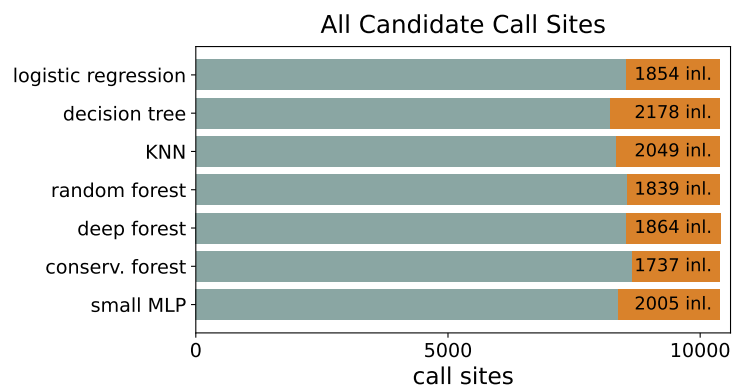


Figure A.5: Student inline/no-inline decisions across candidate call sites.

Bibliography

- [1] M. Godbolt. “Optimizations in C++ Compilers.” In: *ACM Queue* (2019).
- [2] T. Theodoridis, T. Grosser, and Z. Su. “Understanding and Exploiting Optimal Function Inlining.” In: *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS ’22. 2022, pp. 977–989.
- [3] M. Trofin et al. “MLGO: A Machine Learning Guided Compiler Optimizations Framework.” In: *arXiv preprint arXiv:2101.04808* (2021).
- [4] G. Fursin et al. “MILEPOST GCC: Machine Learning Enabled Self-tuning Compiler.” In: *International Journal of Parallel Programming* 39.3 (2011), pp. 296–327.
- [5] LLVM Project. *Machine Learning Guided Optimization*.
- [6] T. V. Marinov, A. Agarwal, and M. Trofin. “Offline Imitation Learning from Multiple Baselines with Applications to Compiler Optimization.” In: *arXiv preprint arXiv:2403.19462* (2024).
- [7] K. D. Cooper and L. Torczon. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.
- [8] S. S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [9] LLVM Project. *LLVM Language Reference Manual*.
- [10] LLVM Project. *LLVM Passes*.
- [11] M. Godbolt. *Inlining - the Ultimate Optimisation*.
- [12] J. Cavazos and M. F. P. O’Boyle. “Automatic Tuning of Inlining Heuristics.” In: *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 2005.
- [13] A. H. Ashouri et al. “MLGOPerf: An ML Guided Inliner to Optimize Performance.” In: *arXiv preprint arXiv:2207.08389* (2022).